

A Strategic Blueprint for Integrating Free Hugging Face Models into HIMARI's Layer-1 Sentiment Analysis

Model Selection and Benchmarking: The Accuracy-Latency Frontier

The foundational objective of achieving a sentiment signal layer with greater than 85% accuracy at a zero-dollar operational cost necessitates a meticulous evaluation of available open-source models from the Hugging Face ecosystem. This process extends beyond a simple comparison of reported metrics; it requires a deep dive into the nuances of model architecture, training data, and performance characteristics under realistic deployment conditions. The initial comparative matrix provides a valuable starting point, identifying several strong candidates, but a comprehensive analysis reveals that model selection is a complex trade-off between predictive power, speed, and domain specialization . The evidence strongly indicates that no single model universally dominates all others; instead, a portfolio of specialized models offers the highest potential for success. For instance, the ProsusAI/finbert model is cited with an accuracy range of 84-86%, making it a prime candidate for analyzing formal financial news . However, its suitability diminishes when faced with the informal, often slang-laden language prevalent on platforms like Reddit and Telegram [6](#) . Conversely, models like curiously/tiny-crypto-sentiment-analysis (78-80% accuracy) and cardiffnlp/twitter-roberta-base-sentiment-latest (85-88% accuracy) are explicitly trained on crypto-related and social media data, respectively, granting them a distinct advantage in understanding the unique linguistic patterns of these sources [6](#) .

A critical insight from the provided research is the profound impact of domain-specific fine-tuning. Studies consistently show that models pre-trained on general corpora experience a significant drop in performance when applied directly to financial contexts. One study demonstrated that zero-shot FinBERT scored only 56.7% accuracy on a multi-sector financial corpus, whereas its fine-tuned version achieved a substantial 71.6% accuracy [14](#) . Similarly, another experiment showed that GPT-4o-mini's accuracy on a financial dataset surged from 77.52% (zero-shot) to 87.79% after being fine-tuned on a combination of the FiQA and Financial PhraseBank datasets [62](#) . This underscores the

necessity of either leveraging models already fine-tuned on financial data or undertaking a targeted fine-tuning effort. The question of whether fine-tuning FinBERT on just 500 crypto headlines is worthwhile is therefore central. Given the documented gains from even smaller datasets in related work, it is highly probable that such an effort would yield a tangible increase in accuracy, potentially in the +2-3% range, which could be critical for crossing the 85% threshold [14](#) [62](#) . The choice of models should therefore prioritize those with proven domain adaptation, such as FinBERT, which was specifically developed for financial texts [39](#) , and specialized variants like CryptoBERT, which has been explicitly fine-tuned on crypto news, Telegram, and social media [6](#) .

Beyond accuracy, latency is a non-negotiable constraint for a production-ready quantitative trading system. The user's requirement of <50ms latency for real-time signals dictates a careful selection of both models and inference infrastructure . The initial matrix correctly identifies DistilRoBERTa-financial as having very low latency (5-10ms), making it an excellent candidate for high-frequency applications . In contrast, multimodal models like TheFinAI/FinLLaVA exhibit higher latency (200ms+), placing them better suited for lower-frequency analytical tasks rather than direct intraday trading signals . The performance of these models is also highly dependent on the underlying hardware. GPU-accelerated inference can deliver 5-20x speedups over CPU-based processing, a crucial consideration for meeting latency targets [18](#) . However, the relationship between quantization and latency is not always linear. While INT8 quantization is frequently cited as a powerful optimization technique to reduce model size and accelerate inference, some empirical benchmarks reveal that it can paradoxically increase latency due to factors like quantization/dequantization overhead or a lack of native hardware acceleration for INT8 operations on certain CPUs [43](#) [44](#) [45](#) . Therefore, any claims of latency reduction through quantization must be validated empirically on the target deployment hardware. The table below synthesizes the available benchmark data, highlighting the nuanced trade-offs between different models.

Model	Domain / Strength	Accuracy (%)	Latency (ms)	Key Considerations
ProsusAI/finbert	Formal News, Financial Texts	84 - 86	20 - 30	High accuracy on structured financial language; may struggle with informal slang. 6 39
DistilRoBERTa-financial	Speed, Efficiency	79 - 82	5 - 10	Extremely low latency makes it ideal for high-frequency signals. Performance depends on quantization and hardware. 40 49
curiously/tiny-crypto	Informal Social Media	78 - 80	40 - 80	Specialized for crypto discourse on Reddit and Telegram, capturing informal language better than general models. 6
cardiffnlp/twitter-roberta	Social Media (Twitter)	85 - 88	12 - 18	Excels on Twitter's unique linguistic style and is a top-performing model for social sentiment analysis. 23 69
FinLLaVA	Multimodal (Text + Chart)	89	200+	Highest accuracy among listed models, but high latency limits its use to non-real-time analysis.
GPT-4o mini (API)	General Purpose LLM	82 (Baseline)	500	Represents a strong API baseline; demonstrates that prompt engineering can achieve high accuracy without full fine-tuning. 38

Further analysis reveals that RoBERTa-based architectures consistently outperform lexicon-based models like VADER and even some fine-tuned BERT variants in certain contexts. A study using a Bi-LSTM with RoBERTa sentiment achieved a lowest Mean Absolute Percentage Error (MAPE) of 2.01% and highest directional accuracy of 79.5% on a large dataset of Bitcoin tweets, significantly outperforming a VADER-Bi-LSTM model which had a MAPE of 3.90% [23](#) . Another study found that rule-based analyzers like VADER and TextBlob were more effective than transformer-based cryptoBERT variants in predicting Bitcoin price movements, suggesting they might capture on-chain slang (e.g., 'pump', 'dump') more effectively than models trained purely on textual corpora [8](#) . This highlights a crucial finding: the optimal approach may involve a hybrid system that combines the strengths of different model families. For example, using a lexicon-based model for detecting specific slang terms and a transformer model for understanding contextual nuance could yield superior results. The predictive power of these models is not merely academic; it translates directly to market impact. Research shows that Twitter sentiment analysis on ZClassic (ZCL) yielded a Pearson correlation of 0.806 with hourly price fluctuations, and a study on Dogecoin found that TikTok video-based sentiment improved short-term price return forecasting by 35% [3](#) [25](#) . These findings validate the core premise of the project and provide a clear path toward developing a profitable sentiment-driven trading strategy.

Dynamic Ensemble Architecture: Optimizing Weighting and Conflict Resolution

While selecting high-performing individual models is a critical first step, the true power of a sophisticated sentiment layer lies in its ability to intelligently combine these disparate signals into a cohesive whole. The initial proposal of a static 50/30/20 weighting scheme—assigning weights of 0.50 to news, 0.30 to crypto-specific discourse, and 0.20 to broader social media—is a logical and defensible starting point. It reflects the perceived relative importance and reliability of different data sources. However, the dynamic nature of financial markets demands a more adaptive approach. Static weights fail to account for shifting market regimes, sudden news events, and changing correlations between different sentiment channels. The provided research strongly advocates for a dynamic weighting scheme, specifically one that balances "Recency vs. Popularity," to create an ensemble that evolves with the market. This approach acknowledges two primary drivers of market sentiment: recency, where the freshest information is often most predictive, and popularity, where a surge in social volume can signal a major event or trend.

The concept of recency is well-supported by empirical evidence. One study on Bitcoin price prediction concluded that using previous-day sentiment was optimal for one-day-ahead forecasts, whereas incorporating current-day sentiment degraded model performance [35](#). Another analysis identified a 3-hour lag between the generation of a sentiment signal and its observable effect on price, reinforcing the idea that a delay between data ingestion and signal utilization is not only acceptable but perhaps necessary for accurate prediction [25](#). A dynamic system that tracks the age of incoming data and adjusts model weights accordingly can capitalize on this temporal dependency. For instance, if a new, high-importance news article is published, the weight assigned to the FinBERT model could be temporarily increased. Conversely, during periods of high retail engagement on social media, the weights for models like `twitter-roberta` or `tiny-crypto` could be boosted to capture the prevailing crowd sentiment more effectively. Frameworks like Freqtrade's adaptive ML-driven strategies, which support real-time retraining of classifiers on streaming data, demonstrate the practical feasibility of implementing such dynamic weighting in a production environment [52](#). This represents a significant upgrade from the initial static configuration, transforming the ensemble from a fixed set of rules into an intelligent, responsive system.

Conflict resolution is another critical aspect of ensemble design. When constituent models produce divergent signals—for example, FinBERT classifying a headline as positive while a crypto-specific model classifies a related social media post as negative—a robust

mechanism is needed to synthesize a single, actionable output. Simple majority voting or averaging can be effective, but more sophisticated methods like stacking, where a meta-model learns how to optimally combine the outputs of base models, have shown superior performance in financial forecasting tasks ²⁹. An empirical study on cryptocurrency price trend forecasting found that a stacking ensemble achieved 81.80% accuracy, outperforming bagging and boosting methods ²⁹. Furthermore, the literature suggests that different models possess inherent biases. One study noted that GPT models excel at identifying positive labels, while FinBERT and standard BERT models are stronger at classifying neutral sentiment ⁶. This implies that a simple weighted average might not be optimal. Instead, a more nuanced conflict resolution strategy could involve calibrating the outputs of each model before aggregation. For instance, if FinBERT is known to be conservative with positive labels, its positive scores could be slightly adjusted upwards, while Twitter-RoBERTa's scores could be calibrated downwards to mitigate hype. This calibration process can be learned from historical data, where the ground truth of subsequent price movements serves as the label for determining the optimal adjustment for each model's output.

The optimal number of models in the ensemble is also a subject of investigation. While a three-model setup (news, crypto-social, and a fast-response model) seems intuitive, there may be diminishing returns from adding more components. A study that combined seven different sentiment signals—including LIWC, VADER, Flair, FinBERT, CryptoBERT, LLaMA-2, and LLaMA-3—found that the unified daily sentiment index performed best, suggesting that a carefully curated ensemble can outperform a larger, less focused one ¹¹. The principle of diversity is key; the models in the ensemble should ideally be complementary, drawing insights from different types of data (e.g., formal news vs. informal tweets) or using different algorithmic approaches. The proposed architecture, routing news to FinBERT, crypto/social to `tiny-crypto`, and broader social to `twitter-roberta`, aligns perfectly with this principle of diversification. Adding a fourth model, such as a lightweight DistilRoBERTa variant for ultra-low-latency signals, could further enhance the ensemble's capabilities, but only if it provides a unique and valuable signal that is not already captured by the existing three models. The expected composite accuracy of the ensemble can be calculated, but it is important to note that it is not simply the weighted sum of individual accuracies. Due to the correlated nature of errors between models, the actual performance will depend heavily on the degree of independence between the constituent models' predictions. A rigorous analysis of the correlation between model outputs is therefore a prerequisite to designing an effective ensemble ^{6 47}. The ultimate goal is to build a system that is not just accurate, but robust and adaptive, capable of navigating the complexities of financial markets by intelligently synthesizing a diverse set of signals.

Data Integration and Preprocessing Pipeline

A state-of-the-art sentiment analysis engine is only as good as the quality of the data it consumes. Building a robust and scalable data pipeline is therefore a foundational task that precedes model selection and feature engineering. The initial research plan correctly identifies the need to catalog free and low-cost data sources, but the provided materials offer a much deeper understanding of the available APIs, their limitations, and the preprocessing challenges inherent in working with real-world financial and social media text. The landscape of data sources is rich, spanning traditional financial news aggregators, social media platforms, and even emerging visual sentiment providers. For news, free and low-cost APIs like CryptoPanic and NewsAPI are viable entry points. For social media, libraries like PRAW for Reddit and access to the Twitter Academic Research product track allow for the collection of vast amounts of user-generated content ⁵. For chart-based sentiment, while programmatic access to TradingView is limited, CoinGecko provides the necessary data to correlate price action with textual sentiment. The key challenge is not just acquiring this data, but structuring the pipeline to handle it efficiently and extract meaningful features. A well-designed architecture would ingest raw data from these sources, pass it through a series of preprocessing stages, and then route it to the appropriate sentiment model based on its origin and type.

The preprocessing stage is arguably the most critical part of the pipeline, as it directly impacts the quality of the input fed to the NLP models. Raw text from social media platforms like Twitter and Reddit is notoriously noisy and presents unique challenges that require specialized handling. A comprehensive preprocessing workflow must address several key areas. First, emoji handling is essential, as emojis carry significant emotional weight and should be converted into descriptive text tags (e.g., 😊 → "[positive-emotion]") to be understood by the sentiment model. Second, normalization of slang, abbreviations, and mentions is crucial. Words like "wagmi" or hashtags like "#btc" need to be mapped to standard tokens. Third, tokenization must be handled carefully; standard word-tokenizers may not work well with the fragmented and informal syntax common in social media. Fourth, cleaning steps like removing URLs, special characters, and excessive whitespace are necessary to reduce noise. Finally, stemming or lemmatization can help consolidate words into their root forms, reducing the vocabulary size and improving model generalization. The entire process must be computationally efficient to handle high-volume streams, potentially involving parallel processing and optimized data structures.

Once the text is cleaned and normalized, the pipeline must address the issue of data duplication. In a continuous stream of news and social updates, the same information is often repeated across multiple sources or posted with minor variations. Storing and

analyzing every duplicate piece of data is wasteful and can introduce bias. To combat this, the pipeline should incorporate a deduplication strategy. One effective method is semantic hashing, which generates a hash for a document based on its meaning rather than its exact wording. Documents with similar hashes are considered duplicates or near-duplicates and can be pruned from the stream . Another approach involves fuzzy matching algorithms that compare strings for similarity above a certain threshold, allowing for the identification and removal of near-duplicates. This ensures that the sentiment analysis models receive a more concise and diverse set of inputs, improving the efficiency and effectiveness of the entire system.

The final component of the data pipeline is routing and importance weighting. After preprocessing and deduplication, the data must be directed to the correct model within the ensemble based on its source and nature . As specified in the initial architecture, news articles from sources like CryptoPanic and NewsAPI should be routed to FinBERT, while posts from Reddit and Telegram should go to the crypto-specialized model, and Twitter/X posts should be sent to Twitter-RoBERTa . This routing logic is fundamental to leveraging the strengths of specialized models. Alongside routing, it is crucial to assign an importance score to each piece of data. Not all news headlines or social media posts are created equal. A breaking news story from a reputable financial outlet carries more weight than a speculative comment on a subreddit. Importance can be determined by several factors, including source credibility, headline prominence, and the volume of engagement the post receives . For example, a headline from a source with a high reputation score (e.g., Bloomberg, Reuters) would be given a higher weight than one from a lesser-known blog. Similarly, a tweet that is retweeted hundreds of times might be flagged as a high-popularity event, warranting a temporary boost in its influence within the sentiment calculation. This importance weighting allows the system to focus on the most relevant and impactful pieces of information, enhancing the predictive power of the final sentiment signal.

Feature Engineering and Signal Composition for Predictive Power

Generating a raw sentiment score from a model is only the first step; the true value of the sentiment layer lies in how this score is transformed into a robust, predictive feature that can be integrated into a quantitative trading model. The process of feature engineering involves normalizing the raw output, applying time-series transformations, and combining it with existing technical indicators in a way that maximizes its predictive

contribution. The initial query regarding normalization highlights a key decision point: should the sentiment score be scaled to $[0, 1]$ via min-max scaling, or should it be standardized to have a mean of zero and a unit variance using a z-score transformation? Both methods have their merits. Min-max scaling ensures the feature remains within a predictable range, which can be beneficial for neural networks that are sensitive to the scale of input data. However, z-scoring is often preferred because it accounts for the statistical distribution of the sentiment values over time, making the feature more robust to outliers and changes in the overall sentiment climate. Given that financial sentiment can vary dramatically during bull and bear markets, z-scoring appears to be the more resilient choice for creating a stable feature.

Once normalized, the sentiment score must be transformed into a time-series representation that captures its dynamic nature. Simply using the raw score from a single timestamp is insufficient, as sentiment is inherently a moving target. A more powerful approach is to compute rolling statistics, such as a moving average over a specific period (e.g., 5, 10, or 20 periods). This smoothing process helps filter out short-term noise and captures the underlying trend in sentiment. For example, a 10-period rolling average of the sentiment score would provide a clearer picture of whether bullish or bearish sentiment is gaining or losing momentum over the past few hours or days. The optimal window size for this rolling average is an empirical parameter that must be determined through backtesting. A shorter window (e.g., 5-period) would make the feature more responsive to recent changes but also more susceptible to random fluctuations, while a longer window (e.g., 20-period) would provide a smoother, more stable signal but with a greater lag. Another advanced technique is to create a volatility-adjusted sentiment metric by multiplying the sentiment score by a measure of price volatility. This acknowledges that the impact of a piece of news or social media chatter is amplified in volatile markets. A sentiment score generated during a calm market may have little predictive power, but the same score generated during a period of high volatility could signal a major turning point. This interaction between sentiment and volatility can be a powerful predictor of future price movements.

The ultimate goal of this feature engineering process is to integrate the sentiment signal with HIMARI's existing suite of 50D technical features. This requires careful analysis of the correlation between the new sentiment features and the existing ones to avoid redundancy and identify synergistic combinations. A correlation matrix should be computed to visualize the relationships between all 50 technical features and the newly engineered sentiment features (e.g., z-scored sentiment, 10-period rolling average, volatility-adjusted sentiment). Features that are highly correlated with each other (e.g., both measuring short-term momentum) can be combined or one can be dropped to reduce dimensionality and prevent multicollinearity in the downstream model. The

analysis should also investigate the lag effects of sentiment on returns. By calculating the cross-correlation between the sentiment signal at time $t-k$ and the asset return at time t , it is possible to determine the optimal lookback period (k) for the sentiment signal. Studies have identified lags ranging from 3 hours for Twitter sentiment affecting price ²⁵ to a day for news sentiment influencing one-day-ahead predictions ³⁵. Identifying this optimal lag is crucial for ensuring that the sentiment feature is truly predictive and not just contemporaneously correlated with price movements. The final step is to normalize the engineered sentiment features using z-score scaling to ensure compatibility with the rest of the feature vector before merging them with the existing 50D technical feature set. This systematic approach transforms a simple sentiment classification into a rich, multi-faceted feature that can provide a genuine edge in a quantitative trading model.

Production Implementation: Achieving Sub-50ms Latency at Scale

Deploying a sentiment analysis system for real-time quantitative trading demands a production-grade architecture designed for maximum throughput and minimal latency. The user's strict requirement of <50ms latency for generating a signal cannot be met with off-the-shelf, unoptimized model serving solutions. It necessitates a deliberate and multi-layered strategy focused on inference optimization, efficient batching, and intelligent caching. The core of this strategy revolves around model quantization, particularly the conversion of models from floating-point precision (FP32 or FP16) to integer precision (INT8). This technique reduces the memory footprint of the model by approximately 75% and accelerates computation, especially on hardware with dedicated integer arithmetic units ^{19 63}. Tools like NVIDIA TensorRT and Intel Extension for PyTorch (IPEX) are industry-standard frameworks that provide robust support for this process, enabling significant speedups of 2x to 5x in many cases ^{18 65}. However, as previously noted, the benefits of quantization are not guaranteed and must be empirically validated. Benchmarks have shown that improper implementation can lead to unexpected latency increases due to quantization overhead or a lack of optimized kernels on the target hardware ^{43 45}. Therefore, a mandatory validation phase is required, where each candidate model (including the highly promising DistilRoBERTa) is benchmarked after applying various quantization techniques to confirm it meets the <50ms latency target under realistic load conditions ⁵⁵.

The choice of the inference server and serving framework is equally critical. FastAPI is a modern Python web framework that is exceptionally well-suited for building high-performance APIs for machine learning models [52](#) [65](#). It supports asynchronous request handling, which is crucial for managing concurrent inference requests without blocking. The server should be designed with a batch endpoint that can accept a list of texts and process them simultaneously. This batching strategy is vital for maximizing GPU utilization and throughput. The optimal batch size is a tunable parameter that represents a trade-off between latency and throughput. A larger batch size increases throughput (more texts processed per second) but can also increase the latency for individual requests. The goal is to find a batch size that keeps the GPU busy without causing unacceptable delays for real-time signals. For CPU-based deployments, the situation is more challenging. While a quantized DistilRoBERTa model can achieve impressive latency figures on modern CPUs (e.g., 1–4 ms per request on Intel Ice Lake Xeon CPUs via Hugging Face Infinity), sustained high-throughput on a single CPU core can still be difficult [49](#). In such scenarios, horizontal scaling by deploying the service across multiple machines or using containerization technologies like Docker and Kubernetes becomes necessary to handle high-volume social media streams.

To further optimize performance and reduce redundant computations, a caching layer is indispensable. A distributed cache like Redis should be deployed to store the sentiment scores of recently analyzed texts. When a new piece of data arrives, the system should first check the cache. If the text (or a semantically similar version) has been processed recently, the cached result can be returned immediately, bypassing the costly inference step. The Time-To-Live (TTL) for the cache entries should be carefully configured. A TTL of 5 minutes, as suggested in the initial architecture, is a reasonable starting point, as sentiment is often most relevant within a short time window. Cache invalidation policies must also be considered; for instance, if a cached piece of news is updated, the old entry in the cache must be invalidated. Monitoring is the final pillar of a robust production implementation. The system must track a suite of critical metrics, including model inference latency percentiles (p50, p95, p99), error rates (inference failures, API timeouts), cache hit rate, and the freshness of the data flowing through the pipeline. These metrics should feed into a centralized monitoring and alerting system (e.g., Prometheus, Grafana) that can automatically trigger alerts when performance degrades or anomalies are detected. This proactive monitoring is essential for maintaining the stability and reliability of the sentiment layer in a live trading environment.

The following table outlines a detailed checklist for the production implementation, integrating the key learnings from the research:

Component	Technology/ Framework	Rationale & Key Considerations
Inference Optimization	NVIDIA TensorRT / IPEX	For INT8/FP16 quantization, kernel fusion, and precision calibration to meet <50ms latency. Mandatory empirical validation required. 18 51 65
Model Serving	FastAPI	Modern, high-performance Python web framework supporting asynchronous requests for low-latency API endpoints. 52 65
Batch Processing	Mini-batch Endpoint	Accepts lists of texts to maximize GPU/CPU throughput. Optimal batch size must be tuned based on hardware and latency requirements. 19
Caching Layer	Redis	Distributed cache to store sentiment scores of recently processed texts, minimizing recomputation. TTL of 5 minutes is recommended. 52
Containerization	Docker / Kubernetes	Enables consistent deployment, scalability, and resource management. Allows for easy horizontal scaling to handle high-volume traffic. 52
Monitoring	Prometheus/ Grafana	Real-time tracking of latency, error rates, cache hit rate, and model accuracy drift. Essential for production health and debugging. 52
CI/CD Pipeline	Automated Retraining	A pipeline to automatically retrain models on new data, ensuring the system adapts to evolving market language and dynamics.

By meticulously executing this production plan, HIMARI can build a sentiment layer that not only meets but exceeds the stringent performance requirements for a competitive quantitative trading platform.

Backtesting, Validation, and Fine-Tuning ROI Analysis

A compelling theoretical framework and a robust production architecture are insufficient without rigorous empirical validation. The final step in evaluating the sentiment layer's viability is a comprehensive backtesting and validation process. This involves using historical data to simulate the performance of a trading strategy that incorporates the sentiment signals and quantifying the resulting improvement over a baseline strategy that relies solely on technical indicators. The provided `danilocorsi/LLMs-Sentiment-Augmented-Bitcoin-Dataset` is an invaluable resource for this purpose, covering the period from 2016 to 2024 and containing both sentiment scores and price data for Bitcoin . The backtesting protocol should follow a strict chronological order to prevent lookahead bias. The dataset should be segmented into training (e.g., 2016-2022), validation (2023), and test (2024) sets . The baseline strategy should be built using only the existing 50D technical feature vector, and its performance should be measured using key metrics such as annualized return, Sharpe ratio, and maximum drawdown. This establishes the performance ceiling that the sentiment layer must surpass.

The incremental addition of sentiment signals is the core of the validation process. The backtest should be run in stages: first, adding the FinBERT-derived news sentiment signal; second, adding the TinyLlama-derived crypto-specific sentiment; and third, adding the Twitter-RoBERTa-derived social sentiment. At each stage, the resulting Sharpe ratio improvement should be calculated and recorded. Based on the preliminary analysis presented in the initial documents, a significant improvement is anticipated. The backtesting report estimates that adding FinBERT news sentiment yields a +13% Sharpe ratio improvement, adding TinyLlama crypto sentiment adds another +10% (for a total of +23%), and finally adding Twitter-RoBERTa social sentiment pushes the total improvement to +27%. This would translate to an expected gain of 0.25-0.35 Sharpe ratio points, which could represent an additional 25,000–35,000 in profit annually on a \$10 million capital base, assuming a risk-free rate of 4% ⁵⁹. However, these numbers must be treated as initial estimates and verified through the formal backtest.

Beyond the Sharpe ratio, other validation techniques are essential to ensure the robustness of the sentiment signal. Walk-forward analysis is a critical method for assessing the model's performance on out-of-sample (OOS) data. This involves rolling a training window forward in time (e.g., 252-day periods corresponding to one trading year) and testing the model on the subsequent period. This simulates how the model would have performed if it were being deployed in the real world and provides a much more realistic estimate of its long-term predictive power than a single, static test set ³. Additionally, a stress test must be conducted to evaluate the model's performance during periods of high market volatility. The backtest should specifically analyze the model's behavior during major market crashes, such as March 2020 and May 2022, to see if the sentiment signal holds up or if it breaks down in extreme conditions. This is crucial for understanding the risk profile of the sentiment-enhanced strategy. Finally, a sensitivity analysis must be performed to determine the break-even accuracy required for the sentiment signal to remain profitable after accounting for transaction costs and slippage ⁶. If the model's accuracy gain is not sufficient to overcome these costs, the strategy will not be viable in practice.

The question of whether to invest in fine-tuning models like FinBERT must be answered with a clear ROI analysis. The cost of fine-tuning includes not only the computational resources required for training but also the cost of creating a high-quality labeled dataset. The initial proposal suggests fine-tuning on 500 crypto headlines, but the cost and quality of this labeling process are unknown. Efficient methods like active learning, where a human annotator is asked to label only the data points that the model is most uncertain about, can significantly reduce labeling costs ⁵⁶. Once a labeled dataset is created, the fine-tuned model should be benchmarked against the baseline version. The expected accuracy gain must then be translated into a Sharpe ratio improvement using the results

from the backtest. The final step is to calculate the break-even timeline: how long would the strategy need to run to generate profits that cover the initial investment in fine-tuning and data labeling? If the ROI is positive and the break-even timeline is reasonable, the investment is justified. The analysis should compare this outcome to the cost-benefit of alternative strategies, such as building a larger ensemble of free models or investing in a proprietary API. The evidence strongly suggests that the marginal gain from fine-tuning is likely to be substantial enough to warrant the investment, especially if it can push the model's accuracy over the critical 85% threshold required for a competitive edge [14](#) [62](#).

Strategic Roadmap for Scope Expansion Beyond Bitcoin

While the immediate objective is to build a high-performance sentiment layer for Bitcoin, the long-term strategic vision for HIMARI must encompass a broader scope to remain relevant and capture value across different asset classes. The user's inquiry about expanding beyond Bitcoin is not merely a technical question but a critical business decision that will define the platform's future. The provided research offers a clear and evidence-based roadmap for this expansion, beginning with a manageable extension to other major cryptocurrencies before venturing into equities and macroeconomic analysis. The first and most logical step is to generalize the sentiment layer from Bitcoin to Ethereum. The rationale for this is supported by strong empirical evidence showing a very high correlation between Bitcoin and Ethereum market dynamics, with a Pearson correlation coefficient of 0.94 [11](#). This suggests that the sentiment signals and predictive models developed for Bitcoin will likely transfer well to Ethereum with minimal modification. Indeed, one study demonstrated that a FinBERT-BiLSTM model achieved exceptional accuracy (97.50% intra-day, 97.36% one-day-ahead) for Ethereum, comparable to its performance on Bitcoin [35](#). This high degree of similarity provides a low-risk entry point for expansion.

Following successful deployment on Ethereum, the next phase of expansion should target other high-correlation assets, with Solana being a prime candidate. The increasing ownership of multiple cryptocurrencies in the U.S. market validates the demand for such a multi-asset capability; in 2025, 49% of American cryptocurrency owners held Ethereum and 18% held Solana, indicating a strong user base interested in altcoins [33](#). However, it is crucial to proceed with caution. The same study that found a high correlation between Bitcoin and Ethereum also noted a much lower correlation between Bitcoin and Litecoin ($r=0.51$) [11](#). This suggests that assets with lower correlation to Bitcoin may require separate, asset-specific sentiment models. A systematic approach would involve analyzing

the correlation of each target asset with Bitcoin and prioritizing those with $r > 0.7$. For assets with lower correlation, the team should conduct a pilot study to assess whether the existing models add any predictive value or if a new, fine-tuned model is required. This phased, data-driven approach minimizes risk and ensures that resources are allocated to assets that will provide the greatest return on investment. The underlying infrastructure, including the data pipelines and feature engineering modules, is largely asset-agnostic and can be reused, making the transition to new assets primarily a matter of data collection and model tuning [26 58](#).

The final horizon for the sentiment layer's expansion is into traditional financial markets, specifically equities. The principles of financial sentiment analysis are fundamentally applicable across asset classes. The FinBERT model, for example, was rigorously tested across 11 different GICS sectors (e.g., Technology, Healthcare, Communication Services) and proved to be a generalizable tool for financial news analysis [14](#). The performance of FinBERT in this multi-sector study was economically significant, yielding Sharpe ratios up to 2.982 for healthcare stocks [14](#). Furthermore, frameworks like the Hybrid Model integrating Graph Neural Networks are explicitly designed to handle the interplay between market sentiment and price dynamics across stocks, commodities, and FX markets, providing a conceptual blueprint for this expansion [26](#). HIMARI's sentiment layer is well-positioned to enter the equities space by adapting its existing infrastructure. This would involve sourcing financial news from equity-focused APIs (e.g., AlphaVantage, Finnhub) and fine-tuning the models on sector-specific financial reports and earnings call transcripts. The tokenizer may also need to be adapted to recognize stock ticker symbols and company-specific terminology.

Finally, to achieve a truly holistic view of market sentiment, the system must evolve to incorporate macroeconomic signals. The Q2 2025 market turmoil, triggered by President Trump's surprise announcement of sweeping tariffs, serves as a powerful illustration of how macroeconomic news can drive cross-asset volatility and sentiment shifts [34](#). The initial data pipeline, which includes news APIs like CryptoPanic and NewsAPI, already captures this type of information. The key is to enhance the routing and weighting logic to identify and prioritize macroeconomic data. This could be achieved by tagging news articles based on their topic (e.g., "tariffs," "interest rates," "GDP") and temporarily increasing the weight of the FinBERT model during periods of high economic uncertainty or volatility. The development of InfoFi platforms like Kaito, Cookie3, and Galxe, which aggregate and reward social media engagement around specific tokens and events, provides a rich source of validated, real-time data that can be fused with on-chain metrics to create a powerful multimodal sentiment signal [34](#). In conclusion, the strategic roadmap for HIMARI's sentiment layer should be a phased expansion. Phase 1 focuses on

extending the Bitcoin model to Ethereum. Phase 2 involves systematically adding other high-correlation altcoins. Phase 3 opens the door to equities, leveraging the proven generalizability of models like FinBERT. And Phase 4 introduces a dedicated module for macroeconomic sentiment, culminating in a comprehensive, multi-asset, multi-modal intelligence platform. This evolution will transform HIMARI from a single-asset tool into a premier provider of market intelligence.

Reference

1. Predict bitcoin values using social sentiments (e.g. news ... https://github.com/pratikpv/predicting_bitcoin_market
2. LLM and NLP models in Cryptocurrency Sentiment ... <https://github.com/Applied-AI-Research-Lab/LLM-and-NLP-models-in-Cryptocurrency-Sentiment-Analysis>
3. Enhancing Cryptocurrency Sentiment Analysis with ... <https://arxiv.org/html/2508.15825v1>
4. priyanshum17/ReplicationStudy <https://github.com/priyanshum17/ReplicationStudy>
5. joshbrook/BTC-Sentiment-Analysis <https://github.com/joshbrook/BTC-Sentiment-Analysis>
6. LLMs and NLP Models in Cryptocurrency Sentiment Analysis <https://www.mdpi.com/2504-2289/8/6/63>
7. siddBhandari/Real-Time-Bitcoin-Price-Change-Prediction- ... <https://github.com/siddBhandari/Real-Time-Bitcoin-Price-Change-Prediction-Using-Twitter-Sentiment-Analysis>
8. Embedded Sentiment in Blockchain Transactional Data <https://arxiv.org/html/2504.13598v1>
9. Do cryptocurrency markets react to issuer sentiments ... <https://www.sciencedirect.com/science/article/abs/pii/S0275531922000447>
10. A swarm-optimization based fusion model of sentiment ... <https://www.nature.com/articles/s41598-025-92563-y>
11. Detecting Bitcoin Sentiment: Leveraging Language Model ... <https://link.springer.com/article/10.1007/s11063-025-11787-1>
12. Financial News Analytics Using Fine-Tuned Llama 2 GPT ... <https://arxiv.org/abs/2308.13032>

13. Fine-Tuning BERT for Sentiment Analysis on Financial News <https://rccpedia.stanford.edu/blog/2024/03/28/fine-tuning-bert-for-sentiment-analysis-on-financial-news/>
14. Fine-Tuning and Explaining FinBERT for Sector-Specific ... <https://www.mdpi.com/2079-9292/14/23/4680>
15. Fine-Tuning GPT-4o Mini for Financial Sentiment Analysis <https://www.analyticsvidhya.com/blog/2024/11/financial-sentiment-analysis/>
16. A Comparative Analysis of Modern Acceleration Frameworks <https://www.mdpi.com/2079-9292/14/15/2977>
17. Performance Testing for AI Applications: Speed & Scalability <https://www.testriq.com/blog/post/performance-testing-for-ai-applications>
18. AI Inference Optimisation: Hardware Acceleration with GPUs <https://www.hyperstack.cloud/blog/case-study/optimising-ai-inference-for-performance-and-efficiency>
19. How to Reduce Model Deployment Costs with Open- ... <https://www.linkedin.com/pulse/how-reduce-model-deployment-costs-open-source-ai-frameworks-6ywof>
20. "Top 10 Edge AI Frameworks for 2025: Best Tools for Real ... <https://blog.huebits.in/top-10-edge-ai-frameworks-for-2025-best-tools-for-real-time-on-device-machine-learning/>
21. Can AI Benchmarks Compare Models Within One Framework ... <https://www.chatbench.org/can-ai-benchmarks-be-used-to-compare-the-performance-of-different-ai-models-within-the-same-framework/>
22. vLLM: Solving Latency & Throughput Challenges in LLMs <https://medium.com/@gsaidheeraj/vllm-solving-latency-and-throughput-challenges-in-large-language-models-32c453dd01ae>
23. Sentiment-driven cryptocurrency forecasting: analyzing LSTM ... <https://link.springer.com/article/10.1007/s13278-025-01463-6>
24. Sentiment-Driven Cryptocurrency Price Prediction <http://www.diva-portal.org/smash/record.jsf?pid=diva2:1815778>
25. Sentiment-Based Prediction of Alternative Cryptocurrency ... <https://www.frontiersin.org/journals/physics/articles/10.3389/fphy.2019.00098/full>
26. A new hybrid approach combining transformer-based ... https://www.sciencedirect.com/science/article/pii/S0952197625032105?dgcid=rss_sd_all
27. Attention-augmented hybrid CNN-LSTM model for social ... <https://www.nature.com/articles/s41598-025-18245-x>
28. Crypto foretell: a novel hybrid attention-correlation based ... <https://link.springer.com/article/10.1186/s40537-025-01291-7>

29. (PDF) A Comparative Analysis of Ensemble-Based Models ... https://www.researchgate.net/publication/392504283_A_Comparative_Analysis_of_Ensemble-Based_Models_for_Predicting_Cryptocurrency_Price_Movements
30. Machine learning approaches to cryptocurrency trading ... <https://link.springer.com/article/10.1007/s44163-025-00519-y>
31. Prediction of cryptocurrency's price using ensemble machine ... <https://www.emerald.com/ejmbe/article/doi/10.1108/EJMBE-08-2023-0244/1249689/Prediction-of-cryptocurrency-s-price-using>
32. Review of deep learning models for crypto price prediction <https://arxiv.org/html/2405.11431v1>
33. 2025 Cryptocurrency Adoption and Consumer Sentiment ... <https://www.security.org/digital-security/cryptocurrency-annual-consumer-report/>
34. Crypto Market Recap: Q2 2025 <https://cryptorank.io/insights/reports/crypto-market-recap-q-2-2025>
35. FinBERT-BiLSTM: A Deep Learning Model for Predicting ... <https://arxiv.org/html/2411.12748v1>
36. Comparative Investigation of GPT and FinBERT's ... <https://www.mdpi.com/2079-9292/14/6/1090>
37. LikithMeruvu/FinBert-Finetuning-for-Stock-Sentiment <https://github.com/LikithMeruvu/FinBert-Finetuning-for-Stock-Sentiment>
38. Financial Sentiment Analysis on News and Reports Using ... <https://ui.adsabs.harvard.edu/abs/arXiv:2410.01987>
39. Financial sentiment analysis using FinBERT with ... <https://arxiv.org/html/2306.02136v2>
40. mrm8488/distilroberta-finetuned-financial-news-sentiment- ... <https://huggingface.co/mrm8488/distilroberta-finetuned-financial-news-sentiment-analysis>
41. How to Quantize a Language Model to int8 Using Optimum <https://medium.com/@hammadhaideerr/how-to-quantize-a-language-model-to-int8-using-optimum-2fa306767338>
42. Distilbert Base Uncased Finetuned Sst 2 English Int8 Static ... https://dataloop.ai/library/model/intel_distilbert-base-uncased-finetuned-sst-2-english-int8-static-inc/
43. INT8 quantized model is very slow #6732 <https://github.com/microsoft/onnxruntime/issues/6732>
44. PTQ quantization int8 is slower than fp16 #1532 <https://github.com/NVIDIA/TensorRT/issues/1532>

45. [Performance] INT8 quantized model run slower than FP32 ... <https://github.com/microsoft/onnxruntime/issues/20052>
46. TheFinAI/fiqa-sentiment-classification · Datasets at ... <https://huggingface.co/datasets/TheFinAI/fiqa-sentiment-classification>
47. FinSoSent: Advancing Financial Market Sentiment Analysis ... <https://www.mdpi.com/2504-2289/8/8/87>
48. FinLlama: LLM-Based Financial Sentiment Analysis for ... <https://dl.acm.org/doi/fullHtml/10.1145/3677052.3698696>
49. Millisecond Latency using Hugging Face Infinity and ... <https://huggingface.co/blog/infinity-cpu-performance>
50. Faster and smaller quantized NLP with Hugging Face and ... <https://medium.com/microsoftazure/faster-and-smaller-quantized-nlp-with-hugging-face-and-onnx-runtime-ec5525473bb7>
51. Improving LLM Inference Latency on CPUs with Model ... <https://towardsdatascience.com/improving-llm-inference-latency-on-cpus-with-model-quantization-28aefb495657/>
52. Top 10 AI-Powered Crypto Trading Repositories on GitHub <https://medium.com/@gwrx2005/top-10-ai-powered-crypto-trading-repositories-on-github-0041862546b6>
53. Bayesian Network Fusion of Large Language Models for ... <https://arxiv.org/html/2510.26484v1>
54. Benchmark MLPerf Inference: Datacenter <https://mlcommons.org/benchmarks/inference-datacenter/>
55. Benchmark MLPerf Inference: Edge <https://mlcommons.org/benchmarks/inference-edge/>
56. Predicting Cryptocurrency's Future Value Using Box- ... <https://www.sciencedirect.com/science/article/pii/S3050700625000799>
57. High-Frequency Cryptocurrency Price Forecasting Using ... <https://www.mdpi.com/2078-2489/16/4/300>
58. A survey of deep learning applications in cryptocurrency - PMC <https://pmc.ncbi.nlm.nih.gov/articles/PMC10726249/>
59. Revisiting Financial Sentiment Analysis: A Language ... <https://arxiv.org/html/2502.14897v1>
60. An Adaptive Multi-Agent Bitcoin Trading System <https://arxiv.org/html/2510.08068v1>
61. The 2024 Conference on Empirical Methods in Natural ... <https://aclanthology.org/events/emnlp-2024/>

62. A Comparative Study of Fine-Tuned Deep Learning Models <https://www.mdpi.com/2227-7072/13/2/75>
63. Foundation models and intelligent decision-making <https://pmc.ncbi.nlm.nih.gov/articles/PMC12169281/>
64. Domain-adaptation Fine-tuning of Foundation Models ... - AWS <https://aws.amazon.com/blogs/machine-learning/domain-adaptation-fine-tuning-of-foundation-models-in-amazon-sagemaker-jumpstart-on-financial-data/>
65. Low-Latency AI Finance: Engineering Speed for DIFC & ... <https://unanimoustech.com/low-latency-ai-finance/>
66. The-FinAI/PIXIU <https://github.com/The-FinAI/PIXIU>
67. PyFin-sentiment: Towards a machine-learning-based ... <https://www.sciencedirect.com/science/article/pii/S2667096823000186>
68. Large Language Models for Nowcasting Cryptocurrency ... <https://www.mdpi.com/2674-1032/4/4/53>
69. Deep learning and NLP in cryptocurrency forecasting <https://www.sciencedirect.com/science/article/pii/S0169207025000147>
70. Pump It: Twitter Sentiment Analysis for Cryptocurrency ... <https://www.mdpi.com/2227-9091/11/9/159>